

SUPPORT DE COURS

SQL analytique et CTE

Fonctions de fenêtre, CTE, récursivité et agrégations avancées. Cours écrit.

MATIÈRE	SQL avancé (PostgreSQL)
ANNÉE SCOLAIRE	2026-2027
INTERVENANT	Hédi Boulord
GROUPE CIBLE	CYBERSEC B2, Ynov Campus Montpellier
SÉANCE	2
DERNIÈRE MISE À JOUR	09/10/2026

1. Pourquoi le SQL analytique ?

En B1, vous avez appris `GROUP BY` : il regroupe les lignes et **écrase le détail**. Si vous calculez le total par région, vous obtenez une ligne par région, et vous perdez les ventes une par une.

Souvent, on veut les deux à la fois : garder chaque ligne **et** y ajouter un calcul fait sur un groupe de lignes (le total de la région, le classement du vendeur, le cumul depuis le début du mois). C'est le rôle des **fonctions de fenêtre** (*window functions*).

Dans ce cours, on voit aussi les **CTE** (`WITH`), qui rendent une requête compliquée lisible, et les **agrégations avancées** (`ROLLUP`, `FILTER`) qui servent à fabriquer des rapports.

Ce que vous saurez faire à la fin : classer, cumuler, comparer une ligne à la précédente, découper une requête en étapes, parcourir une hiérarchie, et produire un tableau de bord avec sous-totaux, le tout en SQL.

Les données du cours

Tous les exemples utilisent deux petites tables, à charger avant de commencer (script complet en annexe, fichier `seance2-donnees.sql`) :

Table	Colonnes
ventes	id, date_vente, vendeur, region, categorie, produit, montant
employes	id, nom, poste, manager_id, salaire

`ventes` contient 24 ventes de juillet à septembre 2026, faites par 4 vendeurs (Hugo et Inès au Sud, Tom et Emma au Nord). Le total est de **3 150 €**.

PostgreSQL pas encore installé ? Suivez les étapes d'installation du TP0 ou du TP1, puis `CREATE DATABASE analytique;` et `\c analytique.`

2. Les fonctions de fenêtre : le principe

Une fonction de fenêtre calcule une valeur **à partir d'un groupe de lignes, sans les regrouper**. On la reconnaît à `OVER (...)`.

```
SELECT id, vendeur, region, montant,
       SUM(montant) OVER (PARTITION BY region) AS total_region
FROM ventes;
```

id	vendeur	region	montant	total_region
1	Hugo	Sud	30.00	1530.00

id	vendeur	region	montant	total_region
2	Inès	Sud	250.00	1530.00
3	Tom	Nord	90.00	1620.00
4	Emma	Nord	60.00	1620.00

Les 24 lignes sont toujours là. Chacune a gagné une colonne `total_region` : la somme de sa région.

Les trois morceaux de `OVER`

Morceau	Rôle	Exemple
<code>PARTITION BY</code>	Découpe les lignes en groupes (comme un <code>GROUP BY</code> qui ne regroupe pas)	<code>PARTITION BY region</code>
<code>ORDER BY</code>	Ordonne les lignes à l'intérieur de chaque groupe	<code>ORDER BY date_vente</code>
<code>cadre (ROWS BETWEEN ...)</code>	Dit quelles lignes autour de la ligne courante entrent dans le calcul	<code>ROWS BETWEEN 2 PRECEDING AND CURRENT ROW</code>

Tous sont facultatifs. `OVER ()` vide veut dire « toutes les lignes du résultat » :

```
-- part de chaque vente dans le chiffre d'affaires total
SELECT vendeur, montant,
       ROUND(100 * montant / SUM(montant) OVER (), 2) AS pct_du_total
FROM ventes;
```

À retenir : `GROUP BY` réduit le nombre de lignes, `OVER` ne le change jamais.

3. Classer : `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`

Quatre fonctions donnent un rang. Elles ne diffèrent que par la façon de traiter les **ex æquo**.

Fonction	Ex æquo	Exemple de suite de rangs
<code>ROW_NUMBER()</code>	Départage arbitrairement, jamais d'égalité	1, 2, 3, 4
<code>RANK()</code>	Même rang, puis saute des numéros	1, 1, 1, 4
<code>DENSE_RANK()</code>	Même rang, sans trou	1, 1, 1, 2
<code>NTILE(n)</code>	Découpe en <code>n</code> tranches égales	1, 1, 2, 2

Exemple sur les ventes à 250 € (il y en a plusieurs, donc des ex æquo) :

```
SELECT vendeur, montant,
       RANK()      OVER (ORDER BY montant DESC) AS rk,
       DENSE_RANK() OVER (ORDER BY montant DESC) AS drk,
       ROW_NUMBER() OVER (ORDER BY montant DESC, id) AS rn
FROM ventes
WHERE id IN (1,2,3,4,5,7,16);
```

vendeur	montant	rk	drk	rn
Inès	250.00	1	1	1
Tom	250.00	1	1	2
Emma	250.00	1	1	3
Hugo	180.00	4	2	4
Tom	90.00	5	3	5
Emma	60.00	6	4	6
Hugo	30.00	7	5	7

Les trois ventes à 250 € ont `RANK = 1`. Le suivant est **4** pour `RANK` (trois lignes sont passées devant), mais **2** pour `DENSE_RANK`.

⚠ **Piège ROW_NUMBER** : en cas d'ex æquo, l'ordre choisi n'est pas garanti. Ajoutez toujours un critère de départage (`ORDER BY montant DESC, id`) pour obtenir le même résultat à chaque exécution.

On peut classer des agrégats

Une fonction de fenêtre s'exécute **après** le `GROUP BY`. On peut donc classer des sommes :

```
SELECT produit, SUM(montant) AS ca,
       RANK() OVER (ORDER BY SUM(montant) DESC) AS rang
FROM ventes
GROUP BY produit;
```

produit	ca	rang
Ecran 27 pouces	1500.00	1
Ecran 24 pouces	540.00	2
Enceinte	480.00	3
Clavier	270.00	4
Casque	240.00	5
Souris	120.00	6

Le grand classique : le « top N par groupe »

Le meilleur vendeur de chaque région. On classe dans chaque région avec `PARTITION BY`, puis on garde le rang 1. Problème : on ne peut pas filtrer sur une fonction de fenêtre dans le `WHERE`. La solution est une CTE (voir la partie 6).

```
WITH classement AS (  
  SELECT region, vendeur, SUM(montant) AS ca,  
         RANK() OVER (PARTITION BY region ORDER BY SUM(montant) DESC) AS rang  
  FROM ventes  
  GROUP BY region, vendeur  
)  
SELECT * FROM classement WHERE rang <= 1;
```

region	vendeur	ca	rang
Nord	Emma	830.00	1
Sud	Inès	860.00	1

Changez `<= 1` en `<= 3` et vous obtenez le top 3 de chaque région.

4. Cumuler et lisser : les agrégats en fenêtre

Toute fonction d'agrégat (`SUM`, `AVG`, `COUNT`, `MIN`, `MAX`) devient une fonction de fenêtre avec `OVER`. Avec un `ORDER BY` dans le `OVER`, le calcul devient **progressif** : c'est la base du **cumul**.

```
WITH par_mois AS (  
  SELECT date_trunc('month', date_vente)::date AS mois, SUM(montant) AS ca  
  FROM ventes  
  GROUP BY 1  
)  
SELECT mois, ca, SUM(ca) OVER (ORDER BY mois) AS cumul  
FROM par_mois;
```

mois	ca	cumul
2026-07-01	1010.00	1010.00
2026-08-01	1040.00	2050.00
2026-09-01	1100.00	3150.00

`date_trunc('month', ...)` ramène une date au premier jour de son mois : c'est l'outil standard pour grouper par mois.

Le cadre : choisir les lignes du calcul

Par défaut, avec un `ORDER BY`, le cadre va du **début du groupe jusqu'à la ligne courante**. On peut le préciser :

Cadre	Signification
<code>ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code>	Du début jusqu'à la ligne courante (cumul)
<code>ROWS BETWEEN 2 PRECEDING AND CURRENT ROW</code>	La ligne courante et les 2 précédentes
<code>ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING</code>	Une ligne avant, une après
<code>ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING</code>	Toutes les lignes du groupe

Une **moyenne mobile** sur les 3 dernières ventes :

```
SELECT date_vente, montant,
       ROUND(AVG(montant) OVER (
         ORDER BY date_vente, id
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS moyenne_3_ventes
FROM ventes;
```

date_vente	montant	moyenne_3_ventes
2026-07-03	30.00	30.00
2026-07-05	250.00	140.00
2026-07-09	90.00	123.33
2026-07-14	60.00	133.33

Sur les deux premières lignes, il n'y a pas 3 ventes disponibles : la fenêtre est plus courte, et c'est normal.

⚠ **Piège ROWS contre RANGE** : sans précision, un `ORDER BY` utilise `RANGE`, qui traite les lignes **ex æquo** comme un seul bloc (elles reçoivent toutes la même valeur de cumul). Pour un cumul ligne à ligne, écrivez `ROWS` explicitement et départagez avec un `id`.

Nommer une fenêtre

Quand plusieurs colonnes utilisent la même fenêtre, on la déclare une fois avec `WINDOW` :

```
SELECT vendeur, montant,
       SUM(montant) OVER w AS cumul_vendeur,
       COUNT(*)      OVER w AS nb_ventes
FROM ventes
WINDOW w AS (PARTITION BY vendeur ORDER BY date_vente, id);
```

5. Regarder les lignes voisines : `LAG`, `LEAD`,

`FIRST_VALUE`, `LAST_VALUE`

Fonction	Retourne
<code>LAG(col)</code>	La valeur de la ligne précédente (<code>NULL</code> pour la première)
<code>LEAD(col)</code>	La valeur de la ligne suivante (<code>NULL</code> pour la dernière)
<code>LAG(col, 2, 0)</code>	2 lignes avant, et <code>0</code> à la place de <code>NULL</code>
<code>FIRST_VALUE(col)</code>	La première valeur de la fenêtre
<code>LAST_VALUE(col)</code>	La dernière valeur de la fenêtre

L'usage numéro un : **l'évolution d'une période à l'autre.**

```
WITH par_mois AS (  
  SELECT date_trunc('month', date_vente)::date AS mois, SUM(montant) AS ca  
  FROM ventes GROUP BY 1  
)  
SELECT mois, ca,  
       LAG(ca) OVER (ORDER BY mois) AS mois_precedent,  
       ROUND(100.0 * (ca - LAG(ca) OVER (ORDER BY mois))  
             / LAG(ca) OVER (ORDER BY mois), 1) AS evolution_pct  
FROM par_mois;
```

mois	ca	mois_precedent	evolution_pct
2026-07-01	1010.00	(vide)	(vide)
2026-08-01	1040.00	1010.00	3.0
2026-09-01	1100.00	1040.00	5.8

Le premier mois n'a pas de précédent : `LAG` renvoie `NULL`, et le calcul d'évolution aussi.

⚠ **Piège** `LAST_VALUE` : avec le cadre par défaut, la « dernière valeur » est celle de la **ligne courante**, pas du groupe entier. Pour avoir le vrai dernier, précisez le cadre :

```
LAST_VALUE(montant) OVER (ORDER BY id  
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
```

6. Les CTE : découper une requête en étapes

Une **CTE** (*Common Table Expression*) est une requête nommée, définie avec `WITH`, que l'on utilise ensuite comme une table. C'est comme donner un nom à une étape de calcul.

```
WITH ca_vendeur AS (  
  SELECT vendeur, SUM(montant) AS ca  
  FROM ventes  
  GROUP BY vendeur  
)  
SELECT vendeur, ca FROM ca_vendeur WHERE ca > 800;
```

Pourquoi s'en servir ?

- **Lisibilité** : on lit la requête de haut en bas, étape par étape, au lieu de sous-requêtes imbriquées.
- **Réutilisation** : on peut utiliser la même CTE plusieurs fois dans la requête finale.
- **Filtrer une fonction de fenêtre** : c'est indispensable (voir plus bas).

Plusieurs CTE à la suite

On les sépare par une virgule, et chacune peut utiliser les précédentes. Exemple : les vendeurs dont le chiffre d'affaires dépasse la moyenne des vendeurs.

```
WITH ca_vendeur AS (  
  SELECT vendeur, SUM(montant) AS ca  
  FROM ventes GROUP BY vendeur  
) ,  
moyenne AS (  
  SELECT AVG(ca) AS moy FROM ca_vendeur  
)  
SELECT v.vendeur, v.ca, ROUND(m.moy, 2) AS moyenne  
FROM ca_vendeur v  
CROSS JOIN moyenne m  
WHERE v.ca > m.moy;
```

vendeur	ca	moyenne
Inès	860.00	787.50
Emma	830.00	787.50
Tom	790.00	787.50

Pourquoi pas de fonction de fenêtre dans le `WHERE` ?

```
SELECT * FROM ventes WHERE ROW_NUMBER() OVER (ORDER BY id) = 1;  
-- ERROR: window functions are not allowed in WHERE
```

L'ordre d'exécution d'une requête est :

1. `FROM` et les jointures
2. `WHERE`
3. `GROUP BY`
4. `HAVING`

5. les fonctions de fenêtre

6. `SELECT`, `DISTINCT`

7. `ORDER BY`, `LIMIT`

Le `WHERE` passe **avant** le calcul des fenêtres : le rang n'existe pas encore. La solution est toujours la même : calculer le rang dans une CTE (ou une sous-requête), puis filtrer dans la requête qui l'utilise. Exemple : la deuxième plus grosse vente de chaque vendeur.

```
WITH classees AS (  
  SELECT vendeur, produit, montant,  
         ROW_NUMBER() OVER (PARTITION BY vendeur ORDER BY montant DESC, date_vente)  
  AS rn  
  FROM ventes  
)  
SELECT vendeur, produit, montant FROM classees WHERE rn = 2;
```

À retenir : on veut filtrer sur un rang, un cumul ou un `LAG` ? On le calcule dans une CTE, on filtre ensuite.

7. Les CTE récursives : parcourir une hiérarchie

Une table qui pointe vers elle-même (un employé a un `manager_id`, une catégorie a un `parent_id`) forme un **arbre**. Pour le parcourir sur un nombre de niveaux inconnu, on utilise `WITH RECURSIVE`.

Une CTE récursive a toujours deux parties reliées par `UNION ALL` :

1. **Le point de départ** (la requête d'ancrage) : les lignes de niveau 0, ici le patron qui n'a pas de manager.
2. **L'étape récursive** : on joint la table avec le résultat déjà obtenu pour descendre d'un niveau, et ça recommence jusqu'à ce qu'il n'y ait plus de ligne.

```
WITH RECURSIVE equipe AS (  
  -- 1. départ : la directrice  
  SELECT id, nom, poste, 0 AS niveau, nom::text AS chemin  
  FROM employes  
  WHERE manager_id IS NULL  
  
  UNION ALL  
  
  -- 2. récursion : les subordonnés directs de ceux déjà trouvés  
  SELECT e.id, e.nom, e.poste, eq.niveau + 1, eq.chemin || ' > ' || e.nom  
  FROM employes e  
  JOIN equipe eq ON e.manager_id = eq.id  
)  
SELECT repeat(' ', niveau) || nom AS organigramme, poste, niveau  
FROM equipe  
ORDER BY chemin;
```

organigramme	poste	niveau
Sophie Laurent	Directrice	0
Léa Moreau	Responsable Nord	1
Emma Faure	Commercial	2
Tom Garnier	Commercial	2
Marc Dubois	Responsable Sud	1
Hugo Petit	Commercial	2
Noah Blanc	Stagiaire	3
Inès Roux	Commercial	2

La colonne `niveau` compte la profondeur, et `chemin` sert à trier l'arbre dans l'ordre de lecture.

⚠ **Piège : la boucle infinie.** Si les données contiennent un cycle (A est le manager de B, qui est le manager de A), la récursion ne s'arrête jamais. Gardez toujours une condition d'arrêt (un `WHERE niveau < 10` par exemple) quand vous n'êtes pas sûr des données.

Autre usage : fabriquer une suite de dates

Pour afficher aussi les **jours sans aucune vente**, il faut d'abord une table qui contient tous les jours. PostgreSQL a une fonction toute prête, `generate_series`, qui évite la récursion :

```
SELECT j::date AS jour, COALESCE(SUM(v.montant), 0) AS ca
FROM generate_series(DATE '2026-09-01', DATE '2026-09-10', INTERVAL '1 day') AS j
LEFT JOIN ventes v ON v.date_vente = j::date
GROUP BY j
ORDER BY j;
```

jour	ca
2026-09-01	90.00
2026-09-02	0
2026-09-03	0
2026-09-04	250.00

Le `LEFT JOIN` garde les jours sans vente, et `COALESCE` remplace le `NULL` par 0. La même série se fait aussi avec une CTE récursive (`SELECT DATE '2026-09-01' UNION ALL SELECT j + 1 FROM jours WHERE j < ...`).

8. Agrégations avancées pour les rapports

FILTER : un agrégat conditionnel

FILTER (WHERE ...) limite un agrégat à certaines lignes, sans toucher aux autres colonnes. Plusieurs agrégats filtrés différemment peuvent cohabiter dans le même **SELECT**.

```
SELECT region,
       SUM(montant) AS total,
       SUM(montant) FILTER (WHERE categorie = 'Ecrans') AS ecrans,
       COUNT(*)     FILTER (WHERE montant >= 100) AS grosses_ventes
FROM ventes
GROUP BY region;
```

region	total	ecrans	grosses_ventes
Nord	1620.00	1110.00	7
Sud	1530.00	930.00	6

C'est aussi la méthode pour faire un **tableau croisé** (*pivot*) : une colonne par mois.

```
SELECT categorie,
       SUM(montant) FILTER (WHERE date_trunc('month', date_vente) = DATE '2026-07-01') AS
juillet,
       SUM(montant) FILTER (WHERE date_trunc('month', date_vente) = DATE '2026-08-01') AS
aout,
       SUM(montant) FILTER (WHERE date_trunc('month', date_vente) = DATE '2026-09-01') AS
septembre
FROM ventes
GROUP BY categorie;
```

categorie	juillet	aout	septembre
Audio	180.00	240.00	300.00
Ecrans	680.00	680.00	680.00
Peripheriques	150.00	120.00	120.00

ROLLUP, **CUBE**, **GROUPING SETS** : les sous-totaux

Dans un rapport, on veut les lignes de détail **et** les sous-totaux **et** le total général. Avec **GROUP BY** seul, il faudrait trois requêtes et des **UNION**. **ROLLUP** le fait en une.

```
SELECT region, categorie, SUM(montant) AS ca
FROM ventes
GROUP BY ROLLUP (region, categorie)
ORDER BY region NULLS LAST, categorie NULLS LAST;
```

region	categorie	ca
Nord	Audio	360.00
Nord	Ecrans	1110.00
Nord	Peripheriques	150.00
Nord	(NULL)	1620.00
Sud	Audio	360.00
Sud	Ecrans	930.00
Sud	Peripheriques	240.00
Sud	(NULL)	1530.00
(NULL)	(NULL)	3150.00

Les `NULL` signalent « tous » : la ligne `Nord, NULL` est le sous-total de Nord, la dernière est le total général.

Option	Produit
<code>ROLLUP (a, b)</code>	Les groupes <code>(a,b)</code> , <code>(a)</code> et le total <code>()</code> : une hiérarchie de sous-totaux
<code>CUBE (a, b)</code>	Toutes les combinaisons : <code>(a,b)</code> , <code>(a)</code> , <code>(b)</code> et <code>()</code>
<code>GROUPING SETS (...)</code>	Exactement les groupes que vous listez

⚠ **Piège : un vrai `NULL` dans les données** se confond avec le `NULL` de sous-total. La fonction `GROUPING(col)` règle le problème : elle vaut **1** si le `NULL` est un sous-total, **0** sinon. Pour des libellés propres, on écrit `CASE WHEN GROUPING(region) = 1 THEN 'TOUTES' ELSE region END`.

Médiane et percentiles

La moyenne est sensible aux valeurs extrêmes, la médiane beaucoup moins. En PostgreSQL :

```
SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY montant) AS mediane,
       ROUND(AVG(montant), 2) AS moyenne
FROM ventes;
-- mediane = 120, moyenne = 131.25
```

9. Et dans MySQL ?

Les fonctions de fenêtre et les CTE (y compris récursives) existent à partir de **MySQL 8.0**. Quelques différences à connaître :

Notion	PostgreSQL	MySQL 8
Agrégat conditionnel	<code>SUM(x) FILTER (WHERE ...)</code>	<code>SUM(CASE WHEN ... THEN x END)</code>
Sous-totaux	<code>GROUP BY ROLLUP (a, b)</code>	<code>GROUP BY a, b WITH ROLLUP</code>
Suite de dates	<code>generate_series(...)</code>	une CTE récursive
Médiane	<code>percentile_cont(0.5) WITHIN GROUP (...)</code>	pas de fonction native

10. Récapitulatif

Je veux...	J'utilise
Garder chaque ligne et ajouter un calcul de groupe	<code>fonction() OVER (PARTITION BY ...)</code>
Classer	<code>ROW_NUMBER</code> , <code>RANK</code> , <code>DENSE_RANK</code> , <code>NTILE</code>
Un cumul, une moyenne mobile	<code>SUM/AVG() OVER (ORDER BY ... ROWS BETWEEN ...)</code>
Comparer à la ligne précédente ou suivante	<code>LAG</code> , <code>LEAD</code>
Filtrer sur un rang ou un cumul	Calculer dans une CTE , filtrer après
Découper une grosse requête	<code>WITH a AS (...), b AS (...)</code>
Parcourir une hiérarchie	<code>WITH RECURSIVE</code>
Un tableau croisé	<code>SUM(...) FILTER (WHERE ...)</code>
Des sous-totaux et un total général	<code>ROLLUP</code> , <code>CUBE</code> , <code>GROUPING SETS</code>

Les 6 erreurs classiques

1. Mettre une fonction de fenêtre dans le `WHERE` (utilisez une CTE).
2. Oublier de départager un `ROW_NUMBER` en cas d'ex æquo.
3. Oublier que `RANK` saute des numéros alors que `DENSE_RANK` non.
4. Utiliser `LAST_VALUE` sans préciser le cadre.
5. Confondre `ROWS` et `RANGE` dans un cumul.
6. Oublier la condition d'arrêt d'une CTE récursive.

Annexe : script de création des données

Le script ci-dessous crée les tables `employes` et `ventes`. Il est aussi fourni dans le fichier `seance2-donnees.sql` (`\i 'chemin/complet/seance2-donnees.sql'` dans `psql`), ou à copier-coller en entier une fois connecté à la base.

```
-- Données du cours séance 2 : SQL analytique et CTE
-- À exécuter dans une base vide (ex : CREATE DATABASE analytique; \c analytique)

DROP TABLE IF EXISTS ventes;
DROP TABLE IF EXISTS employes;

CREATE TABLE employes (
  id          SERIAL PRIMARY KEY,
  nom         VARCHAR(50) NOT NULL,
  poste       VARCHAR(50) NOT NULL,
  manager_id  INT REFERENCES employes(id),
  salaire     NUMERIC(10,2) NOT NULL
);

CREATE TABLE ventes (
  id          SERIAL PRIMARY KEY,
  date_vente  DATE NOT NULL,
  vendeur     VARCHAR(50) NOT NULL,
  region      VARCHAR(20) NOT NULL,
  categorie   VARCHAR(30) NOT NULL,
  produit     VARCHAR(50) NOT NULL,
  montant     NUMERIC(10,2) NOT NULL
);

INSERT INTO employes (nom, poste, manager_id, salaire) VALUES
('Sophie Laurent', 'Directrice',      NULL, 6500),
('Marc Dubois',    'Responsable Sud',  1,   4200),
('Léa Moreau',     'Responsable Nord', 1,   4300),
('Hugo Petit',     'Commercial',       2,   2900),
('Inès Roux',      'Commercial',       2,   3000),
('Tom Garnier',    'Commercial',       3,   2800),
('Emma Faure',     'Commercial',       3,   3100),
('Noah Blanc',     'Stagiaire',        4,   1100);

INSERT INTO ventes (date_vente, vendeur, region, categorie, produit, montant) VALUES
('2026-07-03', 'Hugo', 'Sud', 'Peripheriques', 'Souris', 30.00),
('2026-07-05', 'Inès', 'Sud', 'Ecrans', 'Ecran 27 pouces', 250.00),
('2026-07-09', 'Tom', 'Nord', 'Peripheriques', 'Clavier', 90.00),
('2026-07-14', 'Emma', 'Nord', 'Audio', 'Casque', 60.00),
('2026-07-18', 'Hugo', 'Sud', 'Ecrans', 'Ecran 24 pouces', 180.00),
('2026-07-22', 'Inès', 'Sud', 'Audio', 'Enceinte', 120.00),
('2026-07-27', 'Tom', 'Nord', 'Ecrans', 'Ecran 27 pouces', 250.00),
('2026-07-30', 'Emma', 'Nord', 'Peripheriques', 'Souris', 30.00),
('2026-08-02', 'Hugo', 'Sud', 'Audio', 'Casque', 60.00),
('2026-08-06', 'Inès', 'Sud', 'Peripheriques', 'Clavier', 90.00),
('2026-08-11', 'Tom', 'Nord', 'Ecrans', 'Ecran 24 pouces', 180.00),
('2026-08-15', 'Emma', 'Nord', 'Audio', 'Enceinte', 120.00),
('2026-08-19', 'Hugo', 'Sud', 'Ecrans', 'Ecran 27 pouces', 250.00),
('2026-08-23', 'Inès', 'Sud', 'Peripheriques', 'Souris', 30.00),
('2026-08-26', 'Tom', 'Nord', 'Audio', 'Casque', 60.00),
('2026-08-29', 'Emma', 'Nord', 'Ecrans', 'Ecran 27 pouces', 250.00),
('2026-09-01', 'Hugo', 'Sud', 'Peripheriques', 'Clavier', 90.00),
('2026-09-04', 'Inès', 'Sud', 'Ecrans', 'Ecran 27 pouces', 250.00),
('2026-09-08', 'Tom', 'Nord', 'Peripheriques', 'Souris', 30.00),
('2026-09-12', 'Emma', 'Nord', 'Audio', 'Enceinte', 120.00),
('2026-09-16', 'Hugo', 'Sud', 'Audio', 'Casque', 60.00),
('2026-09-20', 'Inès', 'Sud', 'Audio', 'Enceinte', 120.00),
```

```
('2026-09-24', 'Tom', 'Nord', 'Ecrans', 'Ecran 24 pouces', 180.00),  
( '2026-09-28', 'Emma', 'Nord', 'Ecrans', 'Ecran 27 pouces', 250.00);
```